

CIS Audit Agent

Build an Agent That Audits Linux Hosts Against CIS-Benchmark-Style Rules and Turns Raw Read-Only Command Output Into a Prioritized, Copy-Paste-Ready Remediation Plan

COMMANDS → RULES → LLM → FIX LIST

STUDENT HANDOUT

PART 0 — WHAT THIS PROJECT ACTUALLY IS

0.1 The problem in plain language

A newly provisioned Linux box gets handed to a team. Somewhere on it is a world-writable `/etc/shadow`, an `sshd_config` that still allows `PermitRootLogin yes`, a firewall that was never enabled, or a password policy nobody set. A senior sysadmin could ssh in and check these by hand in ten minutes — but sysadmins are busy, boxes multiply, and the check never happens until an audit or an incident forces it.

Right now this is manual, or it's a 400-line shell script nobody reads that prints PASS/FAIL walls of text nobody triages.

Your job: build an agent that connects to a target host (over SSH, or a local VM/Docker container), runs a fixed set of strictly read-only Linux commands, evaluates the output against roughly ten CIS-Benchmark-style rules, and hands the structured pass/fail data to a language model that turns it into a prioritized, human-readable fix list — each item carrying the exact command needed to remediate it.

The commands your agent runs never change system state. That is the whole safety model: an audit agent that can accidentally `chmod` something or restart a service is not an audit agent — it is an unreviewed change to production. The agent observes; the LLM explains and prioritizes; a human executes the fix.

0.2 What makes this different

Most student scripting exercises stop here: SSH in, run one command, print the output, close the terminal. That is homework.

This project does not end there. You must deliver:

- A running agent. Someone must point it at a real host (or the target VM/container you ship) with nobody from your team present, and get back a structured audit result plus a fix list, on demand.
- Something another team can point at their own box with one config value (host or container name) and one credential, without phoning you for help.
- Proof that every fix-list item traces back to a real command's real output — not a plausible-sounding hardening tip pulled from a blog post.

There are two classic ways to fail this:

- An agent that runs anything that mutates state (a stray `sed -i`, a service restart, an “helpful” `apt install`) breaks the entire audit contract — read-only is not a suggestion.
- A fix list that reads like a generic hardening checklist, disconnected from anything actually observed on the box, is a chatbot cosplaying as an auditor. That is worse than useless — it trains the team to distrust every item your tool produces, including the correct ones.

You are marked on the whole path: target host to prioritized fix list.

PART 1 — IMPORTANT: THE FIX LIST MUST BE GROUNDED

 **READ THIS SECTION CAREFULLY — teams lose marks every time by getting this wrong.**

Your rule engine can be as simple as ~10 hardcoded shell checks with a deterministic pass/fail parser, or — if your organisers permit an LLM/API key — a language model that reads raw command output plus your rule definitions to produce the fix list. Either approach is acceptable, but the LLM's job is to explain and prioritize, never to decide whether a check passed.

What is not acceptable is a fix-list item that could have been generated without ever running a command against the host — generic advice, an invented file path, a “finding” that references a command your agent never ran.

Why this matters

- Trust — a wrong-but-confident fix-list item is worse than no item, because whoever runs the remediation command will trust it.
- Verifiability — anyone reading the report must be able to check each item against the actual captured command output it's based on.
- It costs nothing to skip a rule that passes — if a check passes, silence (or a one-line PASS) is the correct output, not a manufactured nitpick to look busy.

The point of the exercise — wiring “run some commands, print output” is a five-minute task. Grounding every fix-list item in the real, captured stdout of a real read-only command — and being able to show that exact evidence alongside the item — is the skill being taught.

If you use a language model anywhere in the pipeline, you must be able to show, for each fix-list item: the rule ID, the exact command that was run, the raw output (or the relevant excerpt) it was derived from, the pass/fail verdict, and the exact remediation command. An item with no traceable command output scores zero on the grounding component.

Ask your organisers whether an LLM or API key is permitted at this event before assuming you can call one.

PART 2 — BEFORE THE SESSION

2.1 Set up a scratch target

- A disposable VM, or a Docker container, or your own throwaway cloud instance you are allowed to run root-level read checks against. Do not point this at a production or shared machine.
- SSH access (key-based) if targeting a remote host, OR docker exec if targeting a local container — decide which transport you're building for before the clock starts.
- If an LLM is permitted: an API key stored as an environment variable or secret, never in code.

2.2 Pre-pull what you'll need

- Confirm your SSH client library or docker exec wrapper works against a throwaway target at home (paramiko / ssh2 for Python, the ssh2 npm package or a shelled-out OpenSSH client for Node).
- If containerizing the target: pull a base image (ubuntu:22.04 or similar) at home, and pre-bake a few intentional misconfigurations into it so you have something to find on the night.

2.3 Bring test targets

Prepare three targets at home so you're not scrambling on the night:

- A clean/hardened target (should pass most of your ~10 rules) to sanity-check the pipe end to end without drowning in findings.

- A deliberately misconfigured target (root login enabled, weak file permissions, no firewall, blank password policy, etc.) covering most or all of your rule set.
- An unreachable or partially broken target (wrong port, one command missing, permission denied on one check) to test that your agent degrades gracefully instead of crashing the whole run.

2.4 A stack that fits the clock

Layer	Suggested	Note
Trigger	Manual invocation (audit-agent --target host) or a scheduled/cron-style run	No webhook here — this is pull, not push
Transport	SSH (paramiko/Python or ssh2/Node) for remote, docker exec for local container	Pick one as primary; support both only as stretch
Command execution	A fixed, versioned list of read-only commands (an allowlist, not free-form input)	Never build a command by string-interpolating rule-specific input
Rule evaluation	~10 CIS-Benchmark-style checks, each mapped to one or more commands plus a pass/fail parser	Keep parsers boring: regex / exact-match against known-good patterns
LLM layer (if permitted)	One call with structured findings JSON in, structured prioritized fix list JSON out	Prompt only with parsed findings and a raw evidence snippet — never give it a live shell
Output	A single audit report: report.json plus a human-readable report.md / terminal summary	This is what the human executes fixes from

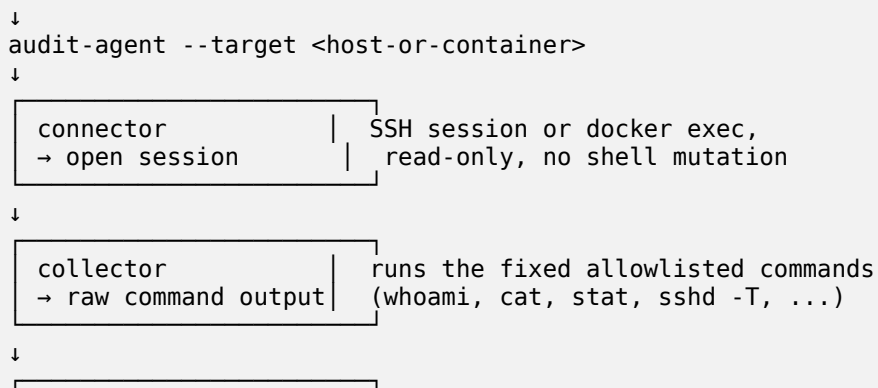
Three notes worth knowing before you start:

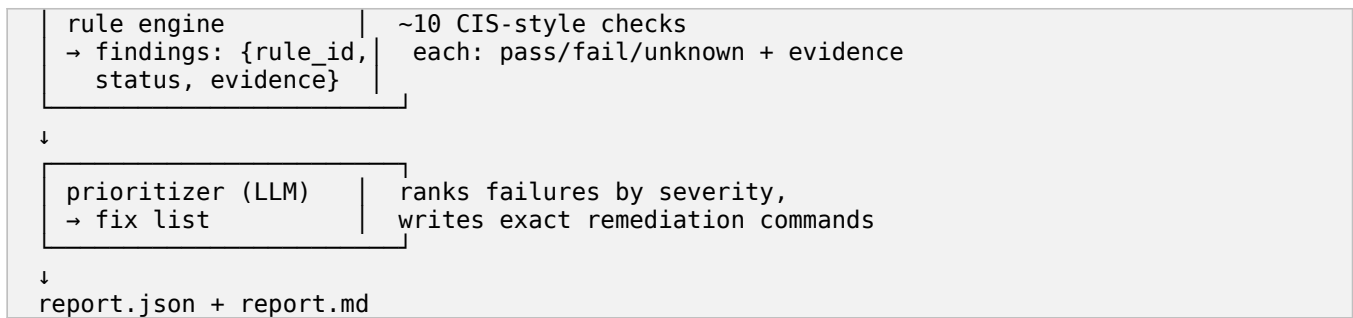
- Command allowlist only — never let the rule engine or the LLM construct or request arbitrary shell commands to run on the target; that's a remote-code-execution hole disguised as a feature.
- Read-only means read-only — no writes, no service restarts, no package installs, not even a “helpful” mutating flag. If a check genuinely needs a mutating command, it doesn't belong in this project — document why you skipped that rule instead.
- SSH host-key verification — don't silently disable strict host-key checking in anything you'd call a real deliverable; if you disabled it for the workshop, say so plainly in the report and why.

PART 3 — WHAT YOU ARE BUILDING

Four moving parts, one endpoint.

Invocation (CLI / scheduled run)





- connector — opens one read-only session, never writes credentials to logs.
- collector — runs the fixed command set, captures stdout, stderr and exit code per command, and tags each with the rule(s) it feeds.
- rule engine — evaluates each of ~10 rules against the relevant command's output, producing PASS / FAIL / UNKNOWN (when a command is unavailable or unreadable) plus the evidence that backs it.
- prioritizer — takes only the structured findings (never raw shell access) and produces a ranked, explained fix list with the exact command to run for each failure.

Why hand the LLM structured findings instead of a raw terminal transcript?

- It keeps every fix-list item traceable to one rule ID and one piece of captured evidence.
- It keeps the LLM from ever seeing — or needing — credentials or a live shell; it only ever sees text you already collected.
- It stops the LLM from inventing a check that was never actually run.

PART 4 — THE INTERFACE CONTRACT

Your agent's internal stages should agree on these shapes so the pieces compose cleanly.

Findings (rule engine → prioritizer)

```
{
  "rule_id": "CIS-5.2.10",
  "title": "SSH root login disabled",
  "command": "sshd -T | grep -i permitrootlogin",
  "status": "FAIL",
  "evidence": "permitrootlogin yes",
  "severity_hint": "high"
}
```

Fix list item (prioritizer → report)

```
{
  "priority": 1,
  "rule_id": "CIS-5.2.10",
  "category": "SSH hardening",
  "finding": "Root login over SSH is permitted (permitrootlogin yes).",
  "why_it_matters": "A leaked or brute-forced root credential grants full remote access with no separate privilege step.",
  "fix_command": "sudo sed -i 's/^PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config && sudo systemctl reload sshd",
  "evidence_ref": "CIS-5.2.10"
}
```

The prioritizer must never mark a rule PASS/FAIL itself — that verdict comes only from the rule engine's evidence. Its job is ranking, explanation, and writing the remediation command.

PART 5 — THE NO-DRIFT RULE (MANDATORY)

Because the agent can be re-run at any time against an unchanged host, a naive pipeline will produce a different-looking report every run — different ordering, different wording, occasionally a different verdict on identical evidence. That is not acceptable in an audit tool.

Before your build freeze, verify:

- Two runs against an unchanged target produce identical PASS/FAIL/UNKNOWN verdicts for every rule.
- Two runs produce the same fix-list ordering, not a reshuffled one — use a low/zero LLM temperature and, for ties, sort by a deterministic key such as `rule_id`.
- The only thing allowed to differ between two identical runs is the timestamp.

⚠ NO PARTIAL CREDIT ON THIS ONE A second run against an unchanged host that reorders or re-words findings scores zero on the reproducibility component of Part 11 — no partial credit.

PART 6 — HOW TO ACTUALLY DO THIS

Read this whole part before coding.

6.1 Keep the rule set boring

Ten CIS-Benchmark-style checks, fixed list, each with a short deterministic parser, e.g.: SSH root login disabled; SSH password authentication disabled in favour of keys; a minimum password length/complexity policy is set; no world-writable files in sensitive system paths; `/etc/passwd` and `/etc/shadow` have correct ownership and permissions; a firewall (`ufw` / `firewalld` / `iptables`) is active; automatic security updates are enabled; no accounts have empty passwords; `sudoers` contains no blanket `NOPASSWD` wildcard; no unexpected service is listening on all interfaces. You are marked on grounded, correctly evidenced findings, not on covering every control in the full CIS benchmark. Widen the list only if you finish early.

6.2 Build the pipe before the logic

This is the single most important paragraph in the document. The most common way to fail is to have a clever prompt at hour two and no pipeline that has ever successfully produced one real report.

Build the plumbing with fake logic first:

1. connector opens a real session against a real target
2. collector runs one real command (e.g. `whoami`) and prints the raw output
3. rule engine returns one hardcoded dummy finding
4. prioritizer step returns that dummy finding wrapped as a one-item fix list

Once you see one real end-to-end report generated from a real target, replace the dummy pieces one at a time. From that moment on, every minute you spend improves something that already works.

6.3 “The script ran” is not “the check is correct”

A non-crashing run only means your steps didn't throw. It does not mean your regex correctly parsed `sshd -T` output on this specific distro and OpenSSH version — a parser can silently match the wrong line and report a PASS that isn't real. Print the raw evidence alongside every verdict during development and eyeball it against what you know is actually configured on your test target.

6.4 Keep credentials out of the repo

SSH keys, passwords, and API keys go into environment variables or a secrets file outside the repo, referenced by name only — never a bare password or key committed to git history. Be ready to explain why: a credential committed to a repo's history is compromised the moment it's pushed, full stop — rotation doesn't undo the exposure window.

6.5 Command allowlisting will drift if you let it

Every command the collector is allowed to run should live in one explicit, versioned list that both you and a reviewer can read top to bottom. If your rule engine ever needs “just one more command,” add it to the allowlist deliberately — never let the LLM, a config file, or rule-specific user input request an arbitrary command at runtime.

6.6 Make it repeatable

Same as Part 5 — a second invocation on an unchanged host must produce the same PASS/FAIL verdicts and the same fix-list order. Judges will run your agent twice.

6.7 Assume hostile or broken input

Before the freeze, throw these at your agent and make sure it responds sensibly instead of crashing or fabricating a result:

- a target where SSH is reachable but one command isn't installed (e.g. no ufw on this distro)
- a target where the auditing user lacks permission to read one file (e.g. /etc/shadow without sudo)
- a target that goes unreachable partway through the run (network drop)
- a target with an unusual locale or output format that would break a naive regex parser
- an LLM call that times out or returns malformed JSON (if you're using one)

A clean UNKNOWN verdict with a logged reason is a pass. A crash, or a fabricated PASS/FAIL with no real evidence behind it, is not.

PART 7 — REQUIREMENTS

7.1 Must-have — this is your pass

#	Requirement
1	The agent connects strictly read-only (SSH or docker exec) — zero mutating commands anywhere in the run
2	Every command comes from a fixed, versioned allowlist — no dynamically constructed commands
3	At least ~10 CIS-style rules are implemented, each producing PASS / FAIL / UNKNOWN plus evidence
4	The fix list is prioritized by severity, and every item includes the exact remediation command
5	Every fix-list item traces to one rule_id and one captured evidence snippet
6	Two runs against an unchanged target produce identical findings and identical fix-list ordering
7	A missing or broken command is skipped with a logged reason, never crashed on
8	Credentials and API keys are read from environment/secrets, never hard-coded
9	The run fails loudly (non-zero exit) if the connector cannot establish a session at all
10	Two fresh invocations against the same target produce the same finding set

7.2 Stretch — this is how you win

- A severity-weighted summary at the top of the report (X critical, Y high, Z medium...)
- A re-audit mode that diffs against a prior report and shows what got fixed versus what's new
- A small deterministic “known-good remediation command” table used to cross-check the LLM’s fix commands for at least a subset of rules
- p95 time from invocation to full report under a stated threshold for a full ~10-rule run, actually measured
- One shared collector interface that works against both a remote SSH host and a local Docker container
- Packaged as a single CLI another team can install and point at a host with one flag, with zero copy-pasted setup

PART 8 — TIMELINE (3 HOURS)

Time	Phase	You are done when
0:00 – 0:10	Read, agree architecture, split work	Everyone knows what they own
0:10 – 0:35	Skeleton — connector + collector + dummy finding end to end	A real target returns one real (dummy) fix-list item
0:35 – 1:05	Real collector — all allowlisted commands run and captured cleanly	A debug dump shows correct raw output for every command on a real target
1:05 – 1:45	Real rule engine — ~10 rules with deterministic parsers	Findings array has real, evidenced PASS/FAIL/UNKNOWN entries
1:45 – 2:15	Real prioritizer — ranked fix list, exact commands, no-drift check	Two runs on the same target produce identical ordering
2:15 – 2:35	Hardening — hostile input, credential hygiene, allowlist review	The Part 10 checklist passes
2:35 – 2:40	BUILD FREEZE — commit and push	Nothing further is edited
2:40 – 3:00	Report	REPORT.md committed

Nominate one person as timekeeper and have them call out each checkpoint aloud.

 **THE 2:40 FREEZE IS HARD** Work that is not committed does not exist.

PART 9 — THE REPORT (20 MINUTES)

One file: REPORT.md, in your repository.

5. What we built — five sentences plus the architecture. State honestly what works and what doesn't.
6. The rule set — for each of the ~10 rules, exactly what is checked, which command(s) it reads, and how the parser decides PASS/FAIL/UNKNOWN.
7. Methods — one line per row: transport chosen vs rejected (SSH vs docker exec); command allowlist design; how you enforced read-only; static vs LLM vs hybrid prioritizer, and why.
8. Results — a table of at least six real audit runs (mix of clean, misconfigured, and broken targets): what was flagged, whether it was correct, and — most important — your explanation of any false positive or false negative.
9. How we worked — planned vs actual at each Part 8 checkpoint, one dead end you abandoned and why.

- 10.Limitations and next steps — be specific. “Handle sudo-restricted evidence gathering safely” earns more than “add more checks.”
- 11.How to run it — exact steps to point this agent at a fresh target, from this section alone.

PART 10 — PRE-FREEZE CHECKLIST

At 2:30, stop adding features and verify every line.

- Connector opens a session read-only — no command in the collector can mutate state
- Every command run comes from the fixed allowlist, nothing constructed dynamically
- All ~10 rules produce PASS/FAIL/UNKNOWN with real captured evidence, verified against a real target, not assumed
- Fix list is ranked by severity and every item carries an exact remediation command
- A second run on an unchanged target adds zero new/changed/reordered findings
- Missing commands and permission errors are skipped with a logged reason, not crashed on
- Credentials come from environment/secrets, not hard-coded
- Every finding is tagged with its rule_id and evidence reference
- REPORT.md committed
- Everything pushed

PART 11 — MARKING

Component	Marks
Connector runs clean, first time, strictly read-only	15
Correct architecture — fixed allowlist, structured findings, no ad-hoc commands	10
Findings correctly evidenced (rule + command + raw output)	15
No-drift / reproducibility on repeated runs	10
Rule coverage — ~10 CIS-style checks all represented	10
Grounding — every fix-list item traceable to real evidence	15
Hostile-input handling	5
Report: methods and decisions	10
Report: results interpretation	6
Report: process and honesty	4
Total	100

A team with a static, pattern-based rule engine, correctly evidenced findings and zero drift between runs will beat a team with an LLM-powered rule engine that occasionally hallucinates a verdict. That is not an accident — it is the point of the exercise.

PART 12 — FREQUENTLY ASKED

Do we need an LLM to pass? No. A purely deterministic rule engine with fixed remediation-command templates for each rule can cover all ten checks and score full marks on grounding, since every item is provably tied to real command output.

Can the prioritizer call a language model? Ask your organisers for the rule in force at this event. If permitted, prompt it only with structured findings and evidence snippets — never give it a live shell or credentials, and require it to return structured JSON, never free-text prose you then have to guess a rule mapping for.

Do we need to support sudo-gated checks (like reading /etc/shadow)? No, unless you want the stretch credit. Checks that work with the audit user's normal read permissions are enough to pass and are far lower-risk than granting the agent elevated access.

What if a command isn't installed on the target? Mark that rule UNKNOWN with a logged reason — don't guess at a verdict GitHub-style tooling didn't actually observe.

Can the LLM just write the fix list from a raw terminal transcript instead of structured findings? No — Requirement 5 is that every item traces to one rule_id and one evidence snippet. A raw transcript handed to the LLM doesn't meet that contract, even if the output looks plausible.

Our rule engine misses some real issues. Are we finished? No, but don't panic — rule coverage is 10 of 100 marks. A rule engine that's narrow but 100% grounded, correctly evidenced, and drift-free across repeated runs scores well. Report the misses honestly — that explanation earns marks.

Good luck — and build the pipe first.